Bloom Filter

Introduction

- In Stream Data Mining, data arrives continuously, sequentially, and at high speeds (e.g., millions of events per second from IoT sensors, clickstreams, or network routers).
- In Stream Data Mining, data arrives continuously at high speeds, in massive volumes, and cannot be completely stored in memory.
- Data streams are potentially infinite; we cannot store the full stream in memory or execute expensive disk-based database lookups for every arriving item.
- The core challenge in stream processing is answering queries (like membership checks) in real-time without unbounded disk or RAM usage.
- A Bloom Filter acts as an ultra-fast, memory-bounded probabilistic gatekeeper at the edge of the stream pipeline to filter out irrelevant or target stream elements instantly.

- It is *probabilistic*, meaning it allows false positives (it might say an item exists when it doesn't), but never false negatives (if it says an item is missing, it is 100% missing).
- Query Returns *False* (0): The element is definitely NOT in the set. (zero false negatives).
- Query Returns *True* (1): The element MAY BE in the set (or it might be a false positive).

How Bloom Filter Works

A Bloom Filter is made up of two components:

1. Bit Array

- A bit array is simply an array containing only 0s and 1s.
- Suppose the bit array size is
- M = 10

Initially, every position is set to 0.

Index	0	1	2	3	4	5	6	7	8	9
Bit	0	0	0	0	0	0	0	0	0	0

- Initially, 0 means: No item has mapped to this position yet.

2. k Independent Hash Functions

- Instead of using one hash function, a Bloom Filter uses **k hash functions.**
- Example: h1(x), h2 (x), h3(x)
- Each hash function converts the input item into a position between **0 and m–1**
- If m= 10, possible indices are 0 1 2 3 4 5 6 7 8 9
- **Adding an Item:** Suppose we want to insert **APPLE**
- **Step 1:** Pass APPLE through all three hash functions.
- Suppose they return

Hash Function	Output
h□(APPLE)	2
h□(APPLE)	5
h□(APPLE)	8

- **Step 2:** Set the corresponding bits to 1.

Before insertion

Index	0	1	2	3	4	5	6	7	8	9
Bit	0	0	0	0	0	0	0	0	0	0

After insertion

Index	0	1	2	3	4	5	6	7	8	9
Bit	0	0	1	0	0	1	0	0	1	0

Only positions **2, 5, and 8** change from **0 → 1**.

Insert Another Item
Now insert: CAT

Hash Function	**Output**
h□(CAT)	1
h□(CAT)	5
h□(CAT)	7

- Notice that position 5 is already 1.
- After inserting CAT,

Index	0	1	2	3	4	5	6	7	8	9
Bit	0	1	1	0	0	1	0	1	1	0

- Bloom Filters **never reset a bit from 1 back to 0** during insertion.
- **Searching an Item:** Suppose we search for APPLE
- Again compute

Hash Function	**Output**
h□	2
h□	5
h□	8

- Now check these positions.

Position	Bit
2	1
5	1
8	1

- Since all bits are 1, Result is **Probably Present**
- Searching Another Item: ORANGE

Hash Function	Output
h□	0
h□	4
h□	9

Check these bits.

Position	Bit
0	0
4	0
9	0

- Since **at least one bit is 0** (in fact, all three are 0), Result is **Definitely Not Present**

Why Multiple Hash Functions?

- Using only one hash function would cause many different elements to map to the same position, increasing collisions.
- Using multiple hash functions:

➢ distributes elements more uniformly,

➢ reduces the probability of false positives,

➢ improves the accuracy of membership testing.

Why Do False Positives Occur?

As you insert more elements (n), more bits in the array become 1.
Eventually, a query for a non-existent element might happen to hash to indices that were already set to 1 by other elements.

False Positive Probability Formula

Given:

- m = size of the bit array
- n = number of inserted elements
- k = number of hash functions

The probability of a false positive (p) is approximately:

$$p \approx \left(1 - e^{-\frac{kn}{m}}\right)^k$$

Optimal Number of Hash Functions (k)

To minimize the false positive rate for a given m and n, the optimal value for k is:

$$k = \left(\frac{m}{n}\right) \ln(2) \approx 0.693 \cdot \left(\frac{m}{n}\right)$$

Strengths and Limitations

Advantages:

- Space Efficient: Uses drastically less memory than hash maps or lists.
- Fast Operations: *O(k)* time complexity for insertions and lookups.
- Privacy Preservation: Does not store the actual data, only hashes

Limitations:

- No Deletions: You cannot simply remove an element (turning a 1 to 0 might corrupt other elements sharing that bit).
- False Positives: Always a small chance of incorrect positive matches.
- Cannot Retrieve Items: You cannot list or iterate through the items inside it.

Why it is essential for Data Streams

Metric	Traditional Set / Hash Map	Bloom Filter
Memory Usage	Scales linearly with item count O(N)	Constant fixed size O(m) regardless of stream volume
Lookup Time	O(1) average, but slow at massive scale	O(k) constant time bound to k hash functions
Deletions	Supported	Standard Bloom filters do not support deletions (flipping a 0 back could erase bits used by other items)

Real-World Applications

1. Web Caching
Check whether a webpage has already been cached.

2. Google Chrome Safe Browsing
Detect malicious URLs before downloading full threat databases.

3. Databases (BigTable, Cassandra, HBase)
Avoid unnecessary disk lookups by checking whether a key is definitely absent.

4. Email Spam Filtering
Quickly test if an email sender appears in known spam lists.

5. Network Routers
Identify duplicate packets and reduce unnecessary processing.

6. Blockchain Systems
Efficiently verify whether transactions or addresses may exist without storing every entry.

7. Streaming Analytics
Detect whether a user, event, or sensor reading has been seen before in real-time data streams.

BLOOM FILTER EXAMPLE (WITH ACTUAL HASH FUNCTION CALCULATIONS)

Given: Bit array size $m = 10$

Number of hash functions $k = 3$

Character Values ($A = 1, B = 2, \ldots, Z = 26$)

Hash Functions:

$h_1(x) = S \bmod 10$

$h_2(x) = (2S + 3) \bmod 10$

$h_3(x) = (3S + 1) \bmod 10$

S = sum of all character values

Initial Bit Array

Index	0	1	2	3	4	5	6	7	8	9
Bits	0	0	0	0	0	0	0	0	0	0

1) Insert APPLE

Letter	A	P	P	L	E
Value	1	16	16	12	5

$S = 1 + 16 + 16 + 12 + 5 = 50$

$h_1 = 50 \bmod 10 = 0$

$h_2 = (2 \times 50 + 3) \bmod 10 = 103 \bmod 10 = 3$

$h_3 = (3 \times 50 + 1) \bmod 10 = 151 \bmod 10 = 1$

Positions: 0, 3, 1

Set these bits to 1

Index	0	1	2	3	4	5	6	7	8	9
Bits	1	1	0	1	0	0	0	0	0	0

2) Insert CAT

Letter	C	A	T
Value	3	1	20

$S = 3 + 1 + 20 = 24$

$h_1 = 24 \bmod 10 = 4$

$h_2 = (2 \times 24 + 3) \bmod 10 = 51 \bmod 10 = 1$

$h_3 = (3 \times 24 + 1) \bmod 10 = 73 \bmod 10 = 3$

Positions: 4, 1, 3

Set these bits to 1

Index	0	1	2	3	4	5	6	7	8	9
Bits	1	1	0	1	1	0	0	0	0	0

3) Insert DOG

Letter	D	O	G
Value	4	15	7

$S = 4 + 15 + 7 = 26$

$h_1 = 26 \bmod 10 = 6$

$h_2 = (2 \times 26 + 3) \bmod 10 = 55 \bmod 10 = 5$

$h_3 = (3 \times 26 + 1) \bmod 10 = 79 \bmod 10 = 9$

Positions: 6, 5, 9

Set these bits to 1

Index	0	1	2	3	4	5	6	7	8	9
Bits	1	1	0	1	1	1	1	0	0	1

Final Bloom Filter

SEARCH OPERATIONS

① Search CAT (already inserted)

$S = 24 \rightarrow h_1 = 4, h_2 = 1, h_3 = 3$

Position	4	1	3
Bit	1	1	1

All bits are 1 → Probably Present (Correct)

② Search COW (not inserted)

Letters: C(3) O(15) W(23) → $S = 41$

$h_1 = 41 \bmod 10 = 1$

$h_2 = (2 \times 41 + 3) \bmod 10 = 85 \bmod 10 = 5$

$h_3 = (3 \times 41 + 1) \bmod 10 = 124 \bmod 10 = 4$

Position	1	5	4
Bit	1	1	1

All bits are 1 → Probably Present (False Positive)

③ Search TIGER (not inserted)

Letters: T(20) I(9) G(7) E(5) R(18)

$S = 20 + 9 + 7 + 5 + 18 = 59$

$h_1 = 59 \bmod 10 = 9$

$h_2 = (2 \times 59 + 3) \bmod 10 = 121 \bmod 10 = 1$

$h_3 = (3 \times 59 + 1) \bmod 10 = 178 \bmod 10 = 8$

Position	9	1	8
Bit	1	1	0

One of the bits is 0 → Definitely Not Present

SUMMARY

Word	Hash Values (h_1, h_2, h_3)	Result
APPLE	(0, 3, 1)	Inserted
CAT	(4, 1, 3)	Inserted
DOG	(6, 5, 9)	Inserted
CAT	(4, 1, 3)	Probably Present (Correct)
COW	(1, 5, 4)	Probably Present (False Positive)
TIGER	(9, 1, 8)	Definitely Not Present